

Side Channel Attack - Inferring LLC size

Software design, algorithms and configuration parameters of the application

The following algorithm was run assuming the cache line size of the system was 64 bytes, which is standard to the Linux lab machines, and personal hardware.

A method was created in C, and did the following:

- Malloc the size of the array being tested
- Increment each array entry to load them into the cache
- Sequentially move through the array, one cache line of 16 integers (64 bytes) at a time.
- Time for each array size to complete a constant number of reads ($64 * 1024 * 1024$), and average the time for each read - If the entire array is small enough to fit in the cache every access should take average-out to the same time, however, if the array does not fit in the cache there should be a noticeable jump in timing.
- The timing was converted to nanoseconds for easy comparison
- The result was returned

A for-loop would test each array size, starting at 1kB, and finishing at around 33MB and print the result in CSV format. Each array size would be double the previous (e.g. 1024, 2048, etc). 33MB was chosen as the upper bound as the LLC size of this machine is 4MB, so testing beyond that is unnecessary.

Experimentation, and issues:

One issue discovered was that if the overall timing was used as the main measuring factor minute differences could not be picked up on as easily, and singular outliers would impact the entire result. The timing was switched to averaging over all steps in nanoseconds, which resulted in much clearer and more stable results. Relying on an average, instead of a single instance of high access time also could act as a mitigating factor for the impact of ongoing cache use in the background.

Doubling the array size would produce the correct result, even if the array sizes were not to the power of 2. The start value was tested from 1020 bytes. 4177920 bytes is less than 4MiB LLC, so the algorithm correctly indicated 8MB+ had a higher average time, indicating the specific LLC could likely be determined with manual testing within the indicated LLC size range of this program. This

Another issue, if the array sizes were not incremented by a factor of two the timing would effectively flatline (Figure 2). The following experimentations address attempts to resolve this.

Alternative methods:

A method tried was using random accessing to prevent the CPU from predicting the next element. The timing difference between random access and sequential access was around 10 ns worth of time calculating the random number, and jumping around the array. The results were stable, but this method did not solve the flatlining issue.

Another method tested was having a separate single array item access time as the measuring factor. After looping through the array, the second array would be tested if it had been evicted from the cache through timing. The results were stable, however, this method did not solve the flatlining issue.

Another method was tried checking each individual array value, looping fully through the array 100 times (accounting for the size of the array) and averaging per element. This showed promising results, however, upon multiple iterations of testing it was found that the results given were likely affected by the accessing of cache lines multiple times and produced unreliable results.

Step experimentations:

The in-use method was experimented with sequentially moving through *all* elements in the array (step of 1 integer). While the results largely did not change, as 16 integers were tested from the same cache line each time the upper end of the array sizes found a reduced time average from 7ns to 4ns through 16 items from the same cachline diluting the average.

It was found that using a step of 10 integers seemed to counteract the flatlining issue and allowed for tests to be run on array sizes outside the power of two. However, the results spiked one array size earlier than the LLC value so could not be relied upon to indicate the specific value. Increasing/decreasing this value to 9/11 resulted in completely incorrect results, so this was seen as an unreliable solution and thus the cache line size of 16 integers was used.

Reads:

The number of reads tested varied from as high as $1024 * 1024 * 1024$ to as little as $1024 * 1024$. While the lower end produced the same results, they were more prone to fluctuations, especially at the higher array sizes. The larger the number of reads, the more stable the results were, but the larger it went the slower the program became to run. The choice was arbitrary, but $64 * 1024 * 1024$ was chosen to be similar to the example documentation and compromise a reasonable runtime time with stable results.

Results measured

Note: The following results were gained from a system running an AMD Ryzen 3 2200G, with a 4MB L3 cache.

The timings of the accesses of each array size, in bytes, were plotted on a graph. The graph should allow the user to infer the LLC size.

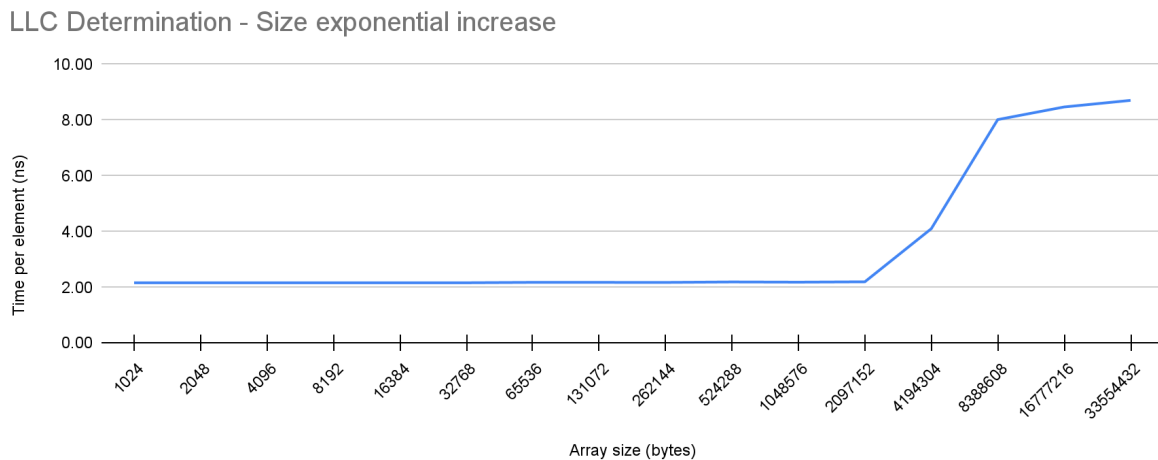


Figure 1: Algorithm results

By increasing array sizes by a factor of two. The timing graph notes that around 4MB is the LLC size of the system's CPU, as the 4MB reading is the first time the read time increases. The 4MB result matches the hardware specifications of the used system.

Flatlining issue

LLC - Size increased by 1.8

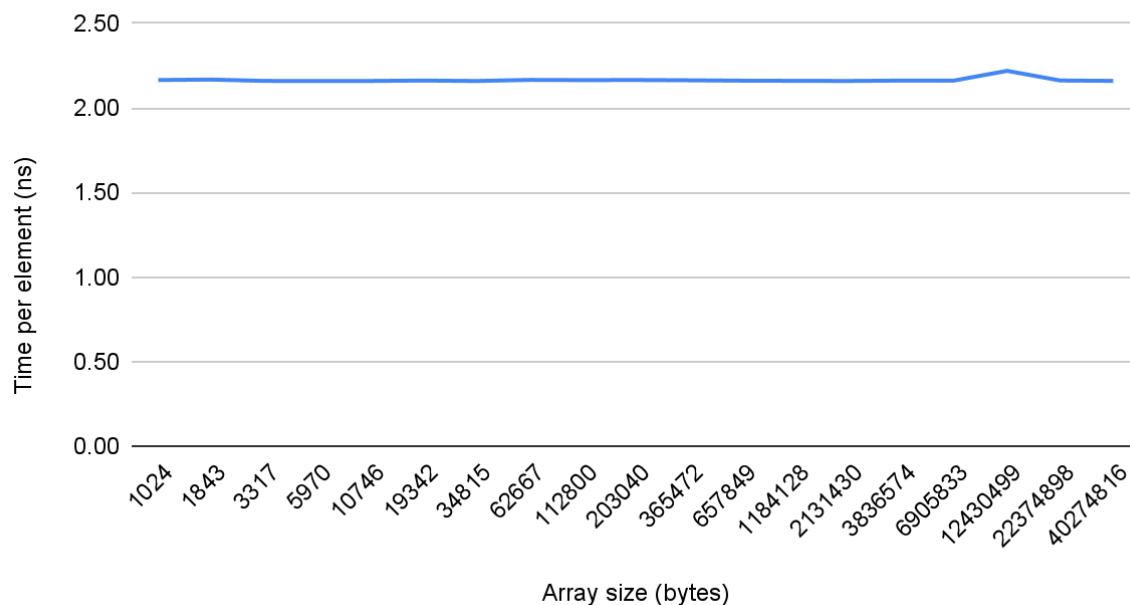


Figure 2: Flatlining timing

A noticeable issue with the method (and all tested methods) was that array sizes not by the power of two produced a flatline effectively. Here is an example of the issue when increasing the array size by 80% (1.8) each time, the average access of all elements was consistent.

There is a minor uptick in the timing around the 12MB array size, however, this is irrelevant to the LLC size.

Discussion on the algorithm's success based on the experimental results.

A noticeable issue with my algorithm was that when measuring produce timings that practically flatline. The exponential increase is consistent with the [example documentation](#)'s examples, so was assumed to be accurate.

While randomly accessing, or testing a separate array item for an evicted status, might be a more reliable method for reducing CPU optimizations as a factor. The simplest method tested moved through each cache line of 64 bytes in a continuously looping fashion, forcing the CPU to continuously load the array into and out of the cache.

The simple sequential results produced clear conclusions, while the ability to accurately find the exact LLC sizing is limited, the general range can be reliably determined by this method. Testing the start value proved that the timing results were accurate, even when one of the array sizes was below the LLC cache size by only 16kB, so manually testing within the specified range a more specific size could be found.